

Bibliothèques statiques et dynamiques en C++

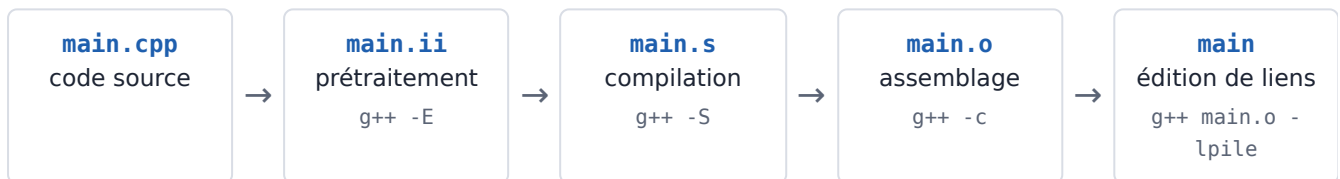
Créer une `.so` et une `.a` sous Linux, les différences entre les deux, et le rôle des fichiers `.hpp`. Exemple utilisé partout dans ce guide : la bibliothèque `libpile` du TP1 (piles génériques + analyseur d'expressions).

1. C'est quoi une bibliothèque ?
2. Statique ou dynamique : les différences
3. Créer une bibliothèque dynamique (`.so`)
4. Créer une bibliothèque statique (`.a`)
5. Templates et fichiers `.hpp`
6. Et sous Windows ? (`.dll` / `.lib`)
7. Aide-mémoire des commandes

1. C'est quoi une bibliothèque ?

Une **bibliothèque** est du code déjà compilé (des fichiers objets `.o`) regroupé dans un seul fichier, pour être réutilisé par plusieurs programmes sans recompiler son code source. Elle est toujours accompagnée de ses **en-têtes** (`.h`) : ils disent au compilateur *ce qui existe* (les déclarations), la bibliothèque contient *le code qui s'exécute*.

Rappel des étapes de compilation d'un programme :



Une bibliothèque intervient à la **dernière étape**, l'édition de liens : le fichier `main.o` contient des appels à des fonctions qu'il ne définit pas (par exemple `PileTableau<int>::empiler`), et l'éditeur de liens va les chercher dans la bibliothèque. La question est alors : **est-ce qu'on copie ce code dans l'exécutable, ou est-ce qu'on le charge au lancement ?** C'est toute la différence entre statique et dynamique.

2. Statique ou dynamique : les différences

Bibliothèque statique `libpile.a`

Le code utilisé est **copié dans l'exécutable** à l'édition de liens.

```
programme
├─ code de main
└─ code de la pile (copié)
```

Au lancement : rien d'autre à charger. Le fichier `.a` n'est plus nécessaire.

Bibliothèque dynamique `libpile.so`

L'exécutable contient seulement une **référence** : « j'ai besoin de `libpile.so` ».

```
programme
├─ code de main
└─ NEEDED : libpile.so
```

`libpile.so` → chargée en mémoire au lancement

Critère	Statique (.a)	Dynamique (.so)
Quand le code est ajouté	À l'édition de liens (copié dans l'exécutable)	Au lancement du programme (par le chargeur <code>ld-linux.so</code>)
Taille de l'exécutable	Plus gros	Plus petit
Bibliothèque nécessaire à l'exécution	Non, l'exécutable est autonome	Oui, sinon le programme ne démarre pas
Mise à jour de la bibliothèque	Il faut refaire l'édition de liens de chaque programme	On remplace le <code>.so</code> , les programmes en profitent sans recompilation
Mémoire	Chaque programme a sa propre copie	Une seule copie en mémoire, partagée entre les programmes
Option de compilation	Rien de spécial	<code>-fPIC</code> pour les <code>.o</code> , <code>-shared</code> pour créer la <code>.so</code>
Outil de création	<code>ar</code> (archiveur)	<code>g++ -shared</code>
Équivalent Windows	<code>.lib</code> (MSVC) / <code>.a</code> (MinGW)	<code>.dll</code>

Mesuré sur le TP1 (même programme `test_expression`)

Lié à `libpile.so` : **27 Ko**. Lié à `libpile.a` : **175 Ko** (le code de la pile et de l'analyseur est copié dedans).

Si on retire `libpile.so`, la version dynamique refuse de démarrer :

```
error while loading shared libraries: libpile.so: cannot open shared object file
La version statique, elle, fonctionne toujours.
```

Comment choisir ? Dynamique quand la bibliothèque est partagée par plusieurs programmes ou doit pouvoir être mise à jour séparément (c'est le cas de presque toutes les bibliothèques système : `libstdc++.so`, `libc.so` ...). Statique quand on veut un exécutable autonome, simple à distribuer, sans risque de « bibliothèque introuvable ».

3. Créer une bibliothèque dynamique (.so)

Organisation des fichiers du TP1 :

```
tpl/
├── include/                ← en-têtes publics (à donner avec la bibliothèque)
│   ├── Pile.h
│   ├── PileTableau.h      PileTableau.hpp
│   ├── PileListe.h        PileListe.hpp
│   └── Expression.h       Expression.hpp
├── src/
│   ├── PileTableau.cpp     PileListe.cpp Expression.cpp ← code de la bibliothèque
│   ├── test_pile.cpp       test_expression.cpp ...      ← programmes clients
└── Makefile
```

Étape 1 — Compiler chaque fichier source en objet avec `-fPIC`

```
mkdir -p obj
g++ -Wall -O2 -Iinclude -fPIC -c src/PileTableau.cpp -o obj/PileTableau.o
g++ -Wall -O2 -Iinclude -fPIC -c src/PileListe.cpp -o obj/PileListe.o
g++ -Wall -O2 -Iinclude -fPIC -c src/Expression.cpp -o obj/Expression.o
```

- `-c` : compiler seulement, sans édition de liens (on obtient un `.o`).
- `-Iinclude` : chercher les `#include "..."` dans le dossier `include/`.
- `-fPIC` (*Position Independent Code*) : une `.so` est chargée à une adresse mémoire différente à chaque fois, le code ne doit donc pas dépendre d'une adresse fixe. **Obligatoire** pour une bibliothèque dynamique.

Étape 2 — Regrouper les objets dans la `.so` avec `-shared`

```
g++ -shared -Wl,-soname,libpile.so obj/PileTableau.o obj/PileListe.o obj/Expression.o -o libpile.so
```

- `-shared` : produire une bibliothèque partagée au lieu d'un exécutable.
- Le nom doit être de la forme `libnom.so`, ici `nom = pile`.
- `-Wl,-soname,libpile.so` (facultatif) : nom enregistré dans la bibliothèque, que les programmes chercheront au lancement.

Étape 3 — Compiler un programme qui utilise la bibliothèque

```
g++ -Wall -O2 -Iinclude src/test_pile.cpp -L. -lpile -Wl,-rpath,'$ORIGIN' -o test_pile
```

- `-L.` : chercher les bibliothèques dans le dossier courant (`.`).
- `-lpile` : lier avec `libpile` — on enlève le préfixe `lib` et l'extension.
- `-Wl,-rpath,'$ORIGIN'` : enregistre dans l'exécutable « cherche la `.so` dans mon propre dossier ». Sans ça, le programme compile mais ne trouve pas la bibliothèque au lancement.

Attention à l'ordre

`-lpile` doit être placé **après** les fichiers sources/objets qui l'utilisent. `g++ -lpile main.cpp` peut provoquer des erreurs `undefined reference`.

Étape 4 — Exécuter

Le chargeur cherche la `.so` dans cet ordre : le `rpath` de l'exécutable, la variable `LD_LIBRARY_PATH`, puis les dossiers système (`/usr/lib ...`). Trois solutions :

```
./test_pile # si compilé avec -rpath (notre cas)
LD_LIBRARY_PATH=. ./test_pile # sinon : indiquer le dossier au lancement
sudo cp libpile.so /usr/local/lib && sudo ldconfig # ou installer dans le système
```

Étape 5 — Vérifier

```
ldd test_pile # bibliothèques dynamiques chargées par le programme
libpile.so => ../../tpl/libpile.so
readelf -d test_pile | grep -E 'NEEDED|RUNPATH'
(NEEDED) Shared library: [libpile.so]
(RUNPATH) Library runpath: [$ORIGIN]
nm -DC --defined-only libpile.so | grep empiler # fonctions exportées par la .so
nm -DC --undefined-only test_expression # fonctions que le programme attend de
la .so
```

Avec le Makefile du TP1

Tout cela est automatisé : `make` construit `libpile.so` puis les programmes, `make run` les lance, `make clean` supprime tout.

4. Créer une bibliothèque statique (.a)

Étape 1 — Compiler en objets (`-fPIC` inutile)

```
g++ -Wall -O2 -Iinclude -c src/PileTableau.cpp -o obj/PileTableau.o
g++ -Wall -O2 -Iinclude -c src/PileListe.cpp -o obj/PileListe.o
g++ -Wall -O2 -Iinclude -c src/Expression.cpp -o obj/Expression.o
```

Étape 2 — Archiver les objets avec `ar`

```
ar rcs libpile.a obj/PileTableau.o obj/PileListe.o obj/Expression.o
ar t libpile.a # liste le contenu de l'archive
PileTableau.o
PileListe.o
Expression.o
```

- Une `.a` est simplement une **archive** (comme un zip) de fichiers `.o`.
- `r` : ajouter/remplacer les fichiers, `c` : créer l'archive si elle n'existe pas, `s` : créer l'index des symboles.

Étape 3 — Compiler le programme

```
g++ -Wall -O2 -Iinclude src/test_expression.cpp -L. -lpile -o test_expression
```

Même commande que pour la `.so`, sans `rpath` (plus rien à chercher au lancement).

Si `libpile.so` et `libpile.a` sont dans le même dossier

`-lpile` choisit la `.so` en priorité. Pour forcer la version statique, donner le nom exact : `-l:libpile.a` (ou directement le chemin du fichier `libpile.a` dans la commande).

Étape 4 — Vérifier

```
ldd test_expression | grep pile      # n'affiche rien : libpile n'est plus une dépendance
./test_expression                   # fonctionne même si libpile.a est supprimée
```

5. Templates et fichiers .hpp

Le problème : un template n'est pas du code

Un template (`template <typename T> class PileTableau`) est un **modèle**. Le compilateur ne produit du code machine que lorsqu'on l'utilise avec un type précis, par exemple

`PileTableau<int>` : c'est l'**instanciation**. Pour instancier, il doit voir le **code complet** des méthodes, pas seulement leur déclaration.

Conséquence : si on met tout le template dans le `.h` (version « header-only »), une bibliothèque compilée à partir de ces en-têtes serait **vide** — rien n'a été instancié, il n'y a aucun code à y mettre.

La solution utilisée : séparer en trois fichiers

Fichier	Contenu	Qui l'inclut / le compile
PileTableau.h	Déclaration de la classe + liste des types déjà compilés dans la bibliothèque (<code>extern template</code>)	Tous les programmes clients
PileTableau.hpp	Code des méthodes du template	La bibliothèque, et un client qui a besoin d'un nouveau type
PileTableau.cpp	Instanciation explicite pour <code>int</code> , <code>double</code> , <code>char</code> , <code>string</code>	Compilé dans <code>libpile.so</code> / <code>libpile.a</code>

C'est quoi un fichier .hpp ?

- `.hpp` = *template implementation*, « implémentation de template ». C'est une **convention de nommage**, pas une extension spéciale : pour le compilateur, c'est un fichier inclus comme un autre (on voit aussi `.ipp`, `.inl`, `.impl.h`).
- Il contient le code des méthodes d'un template, écrit en dehors de la classe :

```
// PileTableau.hpp
template <typename T>
void PileTableau<T>::empiler(const T& x) {
    if (nb == capacite)
        agrandir();
    tab[nb++] = x;
}
```

- **Pourquoi pas `.cpp` ?** Un `.cpp` est compilé tout seul par le Makefile ou l'IDE. Un `.hpp` ne doit **jamais être compilé seul** (il ne produit rien sans instanciation) : il est fait pour être **inclus**. L'extension évite la confusion.
- **Pourquoi pas tout dans le `.h` ?** Le `.h` reste court et lisible (l'interface), et les programmes qui l'incluent ne recompilent pas le code de la pile : ils utilisent celui de la bibliothèque.

Instanciation explicite et extern template

```
// src/PileTableau.cpp – compilé dans la bibliothèque
#include "PileTableau.tpp"
template class PileTableau<int>;          // « génère ici tout le code de PileTableau<int> »
template class PileTableau<std::string>;

// include/PileTableau.h – vu par les programmes clients
extern template class PileTableau<int>;   // « ne le génère pas, il est dans la bibliothèque »
extern template class PileTableau<std::string>;
```

Résultat : le programme client appelle réellement le code de `libpile.so` (vérifiable avec `nm -DC --undefined-only`, qui affiche `U PileTableau<int>::empiler`).

Et pour un type qui n'est pas dans la bibliothèque ?

La bibliothèque ne peut pas connaître tous les types possibles. Pour `PileTableau<Point>` (type défini dans `test_pile.cpp`), le programme inclut le `.tpp` : le compilateur génère alors `PileTableau<Point>` dans le programme lui-même, tandis que `PileTableau<int>` vient toujours de la bibliothèque.

```
#include "PileTableau.h"
#include "PileTableau.tpp"          // nécessaire uniquement pour les types absents de libpile
struct Point { int x, y; };
PileTableau<Point> pp;              // instancié dans le programme
PileTableau<int> pi;               // fourni par libpile.so
```

6. Et sous Windows ? (.dll / .lib)

Un fichier `.so` ne fonctionne **que sous Linux** (format ELF). Windows utilise un autre format (PE) : il faut recompiler les mêmes sources avec un compilateur Windows. Avec MinGW (g++ pour Windows, par exemple via MSYS2) :

```
g++ -shared -Iinclude src/PileTableau.cpp src/PileListe.cpp src/Expression.cpp \
-o libpile.dll -Wl,--out-implib,libpile.dll.a
g++ -Iinclude src/test_pile.cpp -L. -lpile -o test_pile.exe
```

- `libpile.dll` : la bibliothèque dynamique, à placer **dans le même dossier que le .exe** (Windows la cherche là en premier, pas besoin de `rpath`).
- `libpile.dll.a` : la *bibliothèque d'import*, utilisée seulement à l'édition de liens.
- Une DLL C++ compilée avec MinGW n'est utilisable qu'avec MinGW, pas avec Visual Studio (MSVC) : le format interne des noms de fonctions C++ est différent.

Ces commandes Windows n'ont pas été testées sur la machine du TP (pas de compilateur MinGW installé) ; les commandes Linux des sections 3 et 4 ont toutes été exécutées et vérifiées.

7. Aide-mémoire des commandes

Action	Commande
Objet pour une .so	<code>g++ -Iinclude -fPIC -c fichier.cpp -o fichier.o</code>
Créer la .so	<code>g++ -shared *.o -o libnom.so</code>
Créer la .a	<code>ar rcs libnom.a *.o</code>
Lier un programme	<code>g++ -Iinclude main.cpp -L. -lnom -o main</code>
Trouver la .so au lancement	<code>-Wl,-rpath,'\$ORIGIN' ou LD_LIBRARY_PATH=. ./main</code>
Forcer la version statique	<code>-l:libnom.a</code>
Dépendances d'un exécutable	<code>ldd main</code>
Symboles exportés d'une .so	<code>nm -DC --defined-only libnom.so</code>
Contenu d'une .a	<code>ar t libnom.a</code>
Symboles non définis d'un .o	<code>nm -C fichier.o grep ' U '</code>